



Systemes parallèles et distribués

Projet : Parallel Ant Foraging

Javier andres TARAZONA JIMENEZ

Jose Daniel CHACON

Encadrant : JUVIGNY Xavier

École Nationale Supérieure des Techniques Avancées (ENSTA)
Majeure STIC
Mars 2026

Table des matières

1	Introduction	4
1.1	Objectifs de la simulation	4
2	Analyse du problème - Baseline	5
2.1	État de l'art	5
2.2	Profilage et identification des goulots d'étranglement	5
2.3	Résultats	6
2.3.1	Analyse des Résultats	7
3	Vectorisation	9
3.1	Stratégie de Vectorisation	9
3.1.1	Motivation	9
3.1.2	Du layout AoS au layout SoA	9
3.1.3	Maintien du baseline : un binaire, deux layouts	9
3.1.4	Choix d'implémentation : types compacts et seed par fourni	9
3.1.5	Application dans le noyau : <code>advance_time_soa</code>	10
3.1.6	Découplage de <code>SDL_Point</code> hors hot loop et adaptation du rendu	10
3.1.7	Validation et protocole de comparaison	10
3.2	Résultat du Profilage	10
3.3	Analyse des Résultats	11
4	Parallélisation en mémoire partagée (OpenMP)	13
4.1	Stratégie de parallélisation	13
4.1.1	Objectif	13
4.1.2	Choix d'architecture	13
4.1.3	Boucles parallélisées	13
4.1.4	Correctness et absence de data races	13
4.1.5	Réduction sparse et robustesse d'ordre	13
4.1.6	Profilage OpenMP-safe	14
4.2	Résultat du Profilage	14
4.2.1	Protocole expérimental	14
4.2.2	Calcul explicite du speedup mesuré	15
4.2.3	Définition et lecture de l'efficacité	15
4.2.4	Vérifications empiriques de correctitude	17
4.3	Analyse des Résultats	17

4.4	Interprétation	18
5	Parallélisation du code (MPI) — Première façon	19
5.1	Stratégie de parallélisation	19
5.1.1	Objectif	19
5.1.2	Choix d’architecture	19
5.1.3	Répartition du travail	19
5.1.4	Communications et synchronisation	19
5.1.5	Correctness et cohérence	19
5.1.6	Profilage et métriques MPI	20
5.2	Résultat du Profilage	20
5.2.1	Protocole expérimental	20
5.2.2	Scénario de scalabilité (cours)	21
5.2.3	Baseline MPI1 ($K = 1$, sync à chaque itération)	21
5.2.4	Définition et lecture de l’efficacité	22
5.2.5	Impact de <code>mpi_sync_every</code>	23
5.3	Analyse des Résultats	24
5.3.1	Comportement global et scalabilité	24
5.3.2	Lecture “granularité” et coût de communication	24
5.3.3	Lecture de l’efficacité	24
5.3.4	Pourquoi $K4$ augmente avec np (régime memory-bound)	25
5.3.5	Effet de <code>mpi_sync_every</code> (trade-off cohérence / performance)	25
5.3.6	Remarque sur le temps MPI (wall-time)	25
5.4	Interprétation	25
6	Parallélisation du code (MPI) — Deuxième façon	27
6.1	Stratégie de parallélisation	27
6.1.1	Objectif	27
6.1.2	Choix d’architecture	27
6.1.3	Répartition du travail et données locales	27
6.1.4	Communications	27
6.1.5	Politique de migration (baseline)	27
6.2	Résultat du Profilage	27
6.2.1	Protocole expérimental	27
6.2.2	Définition et lecture de l’efficacité	28
6.3	Analyse des Résultats	30
6.3.1	Scalabilité globale	30
6.3.2	Lecture Amdahl	31

6.3.3	Bottlenecks observés	31
6.3.4	Lecture de l'efficacité	31
6.4	Interprétation	31

1 Introduction

1.1 Objectifs de la simulation

L'objectif principal de la simulation est de modéliser le problème de fourragement d'une colonie de fourmis artificielles sur un terrain 2D hétérogène, afin d'identifier des trajets efficaces entre la fourmilière et une source de nourriture. Le cadre retenu suit un modèle d'intelligence en essaim où des agents simples produisent un comportement collectif émergent à partir de règles locales.

Le modèle vise à reproduire une coordination indirecte entre agents via deux types de phéromones : un phéromone d'exploration vers la nourriture (V_1) et un phéromone de retour vers le nid (V_2). Chaque fourmi évolue avec deux états (*unloaded*/*loaded*) et applique une politique exploration–exploitation pilotée par le paramètre ε , avec des transitions d'état lors de l'arrivée sur la nourriture ou sur la fourmilière.

L'environnement a pour objectif de représenter une difficulté de déplacement réaliste : chaque cellule de la carte possède un coût de traversée dans $[0, 1]$, et les zones non franchissables sont traitées comme obstacles. Le paysage est généré de manière fractale afin d'introduire des gradients de difficulté non uniformes et de contraindre la dynamique des déplacements.

Enfin, la simulation poursuit un objectif d'évaluation fonctionnelle au niveau de la colonie : la quantité de nourriture rapportée au nid sert d'indicateur global de performance. Ce cadre constitue la base expérimentale du projet, avant les étapes d'optimisation algorithmique (vectorisation et parallélisation) décrites dans le contexte général.

2 Analyse du problème - Baseline

2.1 État de l'art

L'implémentation actuelle initialise d'abord l'environnement dans `fractal_land.cpp` en construisant une carte fractale par raffinement récursif de sous-grilles. Dans `ant_simu.cpp`, cette carte est ensuite normalisée pour ramener les altitudes dans l'intervalle $[0, 1]$, ce qui fixe le coût de déplacement utilisé par les fourmis pendant la simulation.

La population est ensuite instanciée avec un nombre fixe de fourmis, des positions initiales pseudo-aléatoires et des paramètres globaux du modèle (*exploration*, *bruit*, *évaporation*). La structure `pheromone` (dans `pheromone.hpp`) maintient deux valeurs de phéromones par cellule, un tampon de mise à jour, ainsi que des bordures fantômes marquées à -1 pour empêcher les traversées hors domaine.

À chaque itération, la boucle principale de `ant_simu.cpp` traite les événements SDL puis exécute `advance_time`. Cette routine appelle `ant::advance` pour chaque agent, applique l'évaporation globale des phéromones (`do_evaporation`) puis valide l'état suivant de la carte (`update`). L'évolution individuelle d'une fourmi, définie dans `ant.cpp`, combine un choix local de voisin (aléatoire ou guidé par le maximum de phéromone), l'accumulation d'un budget de mouvement (`consumed_time < 1`), le marquage local des phéromones et la mise à jour de l'état chargé/non chargé.

Le rendu de l'état simulé est assuré par `render.cpp` et `window.cpp`. Le rendu affiche le terrain, la position des fourmis, la distribution des deux phéromones et la courbe de nourriture cumulée. La couche fenêtre/render SDL est construite avec mécanismes de repli (*fallback*) pour conserver l'exécution même lorsque certaines options matérielles ne sont pas disponibles.

2.2 Profilage et identification des goulots d'étranglement

Le profilage a pour objectif d'isoler les coûts dominants de la simulation avant toute optimisation, en particulier avant les étapes de vectorisation et de parallélisation. Nous présentons ici la définition des blocs à mesurer, instrumentation temporelle, protocole d'exécution reproductible et structure des résultats attendus. Cette section décrit donc la méthode de profilage et les objets techniques mobilisés, sans présenter de résultats chiffrés.

TABLE 1 – Découpage des parties mesurées pour le profilage de la simulation

ID	Nature	Rôle	Périmètre
P0	Initialisation	Génération fractale du terrain	One-shot (démarrage)
P1	Initialisation	Normalisation des coûts du terrain	One-shot (démarrage)
P2	Initialisation	Création et placement de la population	One-shot (démarrage)
K0	Noyau simulation	Coût total du noyau par itération	Itératif
K1	Noyau simulation	Avance globale de toutes les fourmis	Itératif
K2	Noyau simulation	Logique interne hors marquage phéromones	Itératif
K3	Noyau simulation	Marquage local des phéromones	Itératif
K4	Noyau simulation	Évaporation globale des phéromones	Itératif
K5	Noyau simulation	Validation/swap de la carte de phéromones	Itératif
R0	Rendu SDL	Affichage (terrain, fourmis, phéromones, courbe)	Itératif, hors noyau
E0	Événements SDL	Traitement de <code>SDL_PollEvent</code>	Itératif, hors noyau

Les relations structurelles utilisées pour l’analyse des goulots sont explicites :

$$K0 = K1 + K4 + K5$$

$$K1 = K2 + K3 + \text{reste}(K1)$$

Les blocs $P0$, $P1$ et $P2$ sont des coûts *one-shot* d’initialisation et ne doivent pas être mélangés aux métriques par itération. Les blocs $R0$ (rendu SDL) et $E0$ (gestion des événements SDL) sont mesurés séparément et traités comme non parallélisables dans l’analyse du noyau de calcul.

La stratégie instrumentale côté C++ s’appuie sur `timing_profile.hpp` et `timing_profile.cpp`. Le profilage est désormais *instance-based* (plus d’état mutable global partagé) : chaque exécution benchmark manipule son objet `TimingProfile` et les compteurs d’itération `IterTimingNs`. `timing_profile.hpp` fournit l’horloge monotone (`profile_now_ns`) utilisée pour les frontières de kernels ($K0$, $K1$, $K4$, $K5$), tandis que $K2/K3$ sont accumulés côté boucle agents comme temps de travail (*work-time*) dans les métriques d’itération. Cette organisation sépare proprement coûts d’initialisation, coûts noyau et coûts hors noyau.

L’automatisation est assurée par `scripts/measure_temps.sh`. Le pipeline suit une séquence reproductible : compilation release, puis 5 répétitions d’exécution avec 1200 itérations par répétition, 200 itérations de *warm-up* exclues et 1000 itérations effectivement mesurées, en configuration `OMP_NUM_THREADS=1`. Le script consolide les artefacts méthodologiques (`raw_measurements.csv`, `init_times.csv`, `table_a.csv`, `table_b.csv`, `amdahl_prediction.csv`, `summary.md`) afin de préparer la lecture des goulots. Les artefacts générés sont :

- `raw_measurements.csv` : mesures brutes par répétition (compteurs en ns et valeurs dérivées par itération).
- `init_times.csv` : synthèse des coûts *one-shot* d’initialisation ($P0$, $P1$, $P2$).
- `table_a.csv` : métriques des blocs de profilage (moyenne, dispersion et part relative du noyau).
- `table_b.csv` : indicateurs globaux par répétition (T_{iter} , T_{kernel} , T_{render} , débit).
- `amdahl_prediction.csv` : valeurs théoriques de speedup issues du modèle d’Amdahl à partir de la fraction parallélisable estimée.
- `summary.md` : synthèse lisible du protocole et des tableaux générés.

La méthode de lecture priorise les métriques `ms/iter` et % noyau pour identifier les *hotspots* dominants. Les conclusions doivent d’abord cibler les blocs du noyau ($K0$ – $K5$), puis isoler l’impact des composants hors noyau ($R0$, $E0$) pour éviter les biais d’interprétation.

2.3 Résultats

Les résultats présentés ci-dessous proviennent de la campagne de mesure :

`results/test_time_measurements_local2/aos/latest/summary.md`. La configuration utilisée est la suivante : 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 2 – Temps one-shot d’initialisation

Partie	ms moyen	écart-type
P0	8.222398	1.107389
P1	0.830145	0.541537
P2	0.171107	0.032904

TABLE 3 – Décomposition des coûts (profilage de base)

Partie	ms/iter moyen	écart-type	% kernel
K1	7.583137	0.803269	94.3004
K2	3.278775	0.344471	40.7733
K3	2.662628	0.413677	33.1112
K4	0.447192	0.087824	5.5611
K5	0.010333	0.002728	0.1285
R0	2.190812	0.341754	n/a
E0	0.005376	0.000923	n/a

TABLE 4 – Indicateurs globaux par répétition

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	8.619821	6.832405	1.783345	731806.773612
2	11.077200	8.649231	2.422158	578086.088303
3	9.838528	7.800593	2.032789	640976.879203
4	9.514154	7.527849	1.981301	664200.401132
5	12.138599	9.397280	2.734469	532068.839698
Moyenne	10.237660	8.041472	2.190812	629427.796390
Écart-type	1.234388	0.893450	0.341754	69193.508032

Fraction parallélisable candidate :

$$F = (K1 + K4)/K0 = 0.997839$$

TABLE 5 – Prédiction théorique selon Amdahl

Threads	S(p)
2	1.995687
4	3.974235
8	7.880787
16	15.497644

2.3.1 Analyse des Résultats

Cette sous-section interprète les mesures présentées dans les tableaux 2, 3, 4 et 5, ainsi que les indicateurs F et $S(p)$. L'objectif est d'identifier les priorités de parallélisation à partir des données observées, en séparant clairement ce qui relève des coûts d'initialisation, du noyau de simulation et des bornes théoriques de gain.

1. **Coûts d'initialisation ($P0$ – $P2$).** Dans le tableau 2, $P0$ domine très largement la phase d'initialisation (8.222398 ms), alors que $P1$ (0.830145 ms) et $P2$ (0.171107 ms) restent faibles. Cette hiérarchie est cohérente avec le fait que la génération fractale du terrain est la tâche la plus lourde au démarrage. En revanche, comme $P0$ – $P2$ sont des coûts *one-shot* hors boucle itérative, ils ne constituent pas la priorité principale pour améliorer le temps de simulation par itération.
2. **Priorité de parallélisation au niveau noyau.** D'après le tableau 3, $K1$ représente 94.3004% du temps noyau, ce qui en fait la cible prioritaire. Le raisonnement est direct : réduire $K1$ a

l'impact potentiel maximal sur $K0$, avant $K4$ (évaporation) et $K5$ (mise à jour de la carte de phéromones), dont les contributions relatives sont nettement plus faibles.

3. **Décomposition interne de $K1$: $K2$ avant $K3$.** Toujours dans le tableau 3, $K2$ pèse 40.7733% du noyau contre 33.1112% pour $K3$. L'ordre de priorité est donc de paralléliser d'abord $K2$, puis $K3$. Cela confirme que la logique de déplacement/décision des fourmis est plus coûteuse que le marquage des phéromones.
4. **Stabilité expérimentale inter-répétitions.** Le tableau 4 montre une faible dispersion entre les 5 répétitions ($T_{\text{iter_total}}$, T_{kernel} , T_{render} , fourmis/s), ce qui indique une bonne stabilité de la campagne de mesure. Cette stabilité provient du protocole (phase de *warm-up* exclue et répétitions multiples), et non d'un effet spécifique "à partir de la cinquième itération".
5. **Interprétation de la fraction parallélisable F .** La valeur $F = 0.998797$, proche de 1, n'est pas anormale dans ce cadre : la définition $F = (K1 + K4)/K0$ inclut les blocs dominants du noyau et exclut les parties hors noyau ($R0$, $E0$). Cette valeur doit toutefois rester interprétée comme une fraction candidate théorique pour le noyau, et non comme une garantie de speedup réel, car les surcoûts de synchronisation, d'accès mémoire et d'équilibrage peuvent réduire le gain observé.
6. **Lecture correcte de la prédiction d'Amdahl.** Le tableau 5 montre un potentiel de gain important avec l'augmentation du nombre de threads. En particulier, $S(16) = 15.497644$ correspond à un facteur multiplicatif d'environ $\times 15.7$, et non à 15%. Cette tendance soutient clairement la stratégie de parallélisation ; la suite du projet doit donc viser d'abord $K1$, puis $K4$, pour se rapprocher de ce plafond théorique.

En synthèse, les deux *hotspots* centraux sont $K1$ au niveau global et $K2$ à l'intérieur de $K1$. Les mesures actuelles établissent une base robuste pour prioriser les optimisations, mais l'écart entre potentiel théorique et performance finale devra être vérifié expérimentalement après implémentation de la parallélisation. La feuille de route est donc cohérente : concentrer l'effort d'abord sur les blocs dominants du noyau afin de maximiser le retour de chaque évolution de code.

3 Vectorisation

3.1 Stratégie de Vectorisation

3.1.1 Motivation

Le point 5.1 de l'énoncé définit la *vectorisation* comme un changement de représentation des agents : au lieu de manipuler une liste d'objets « fourmi », on stocke séparément les attributs (coordonnées, état, graine de générateur pseudo-aléatoire, etc.) dans des tableaux. Une fourmi est alors repérée uniquement par son indice i dans ces tableaux.

Cette étape est motivée par l'analyse *baseline* : le bloc $K1$ (avance de l'ensemble des fourmis) concentre l'essentiel du temps noyau. L'objectif principal est donc d'améliorer l'efficacité mémoire de cette boucle (localité, cache, bande passante) afin de réduire le coût par itération, tout en préparant la suite (parallélisation OpenMP au point 5.2 de la consigne).

3.1.2 Du layout AoS au layout SoA

Dans la version *baseline*, la population est stockée sous la forme d'un `std::vector<ant>` (*Array of Structs*, AoS). Chaque instance `ant` regroupe plusieurs champs hétérogènes (position `SDL_Point`, état, graine RNG, etc.). Lors de l'itération sur toutes les fourmis, le noyau lit/écrit principalement la position, l'état et la graine, mais les lignes de cache chargées contiennent aussi des données non nécessaires au pas courant.

Nous introduisons donc un layout *Structure of Arrays* (SoA), plus favorable à la localité et à la vectorisation au sens « données » du sujet :

$$\text{AoS} : \{(x_i, y_i, s_i, g_i)\}_{i=0}^{N-1} \longrightarrow \text{SoA} : (x[], y[], s[], g[]).$$

Ainsi, les accès séquentiels `x[i]`, `y[i]`, `state[i]`, `seed[i]` deviennent contigus en mémoire, ce qui augmente la proportion de données utiles par ligne de cache et facilite les boucles par indice.

3.1.3 Maintien du baseline : un binaire, deux layouts

Afin de conserver une référence expérimentale stable, le chemin AoS est maintenu et reste le mode d'exécution par défaut. Le programme expose un paramètre de ligne de commande permettant de choisir la représentation :

- `-layout aos` : exécution *baseline* (`std::vector<ant>`);
- `-layout soa` : exécution vectorisée (`AntsSoA`).

Ce choix « 1 binaire, 2 layouts » évite de dupliquer le projet en deux répertoires (et donc de diverger au fil des modifications) et garantit une comparaison directe AoS vs SoA à paramètres identiques.

3.1.4 Choix d'implémentation : types compacts et seed par fourmi

En SoA, nous stockons explicitement les attributs minimaux requis par la dynamique :

- `x[i]`, `y[i]` en `int32_t` (coordonnées);
- `state[i]` en `uint8_t` (0 = *unloaded*, 1 = *loaded*);
- `seed[i]` en `uint32_t` (état RNG local).

Le choix de types compacts réduit l’empreinte mémoire par agent et améliore l’utilisation du cache, ce qui est particulièrement pertinent puisque la boucle $K1$ parcourt plusieurs milliers de fourmis.

Un point important pour la reproductibilité est la gestion de la graine RNG par fourmi. Dans le code AoS *baseline*, le champ `m_seed` est utilisé dans `ant::advance (src/ant.cpp)` mais n’était pas initialisé dans le constructeur (`src/ant.hpp`), ce qui induit un *comportement indéfini* et rend le baseline instable. La vectorisation impose de rendre cette donnée explicite : chaque fourmi possède sa graine `seed[i]` et les fonctions de génération aléatoire la mettent à jour par référence. Pour garder une comparabilité maximale, la formule du générateur est conservée ; seule l’initialisation devient correcte et déterministe.

3.1.5 Application dans le noyau : `advance_time_soa`

La version SoA introduit un noyau `advance_time_soa` (analogue à `advance_time` dans `src/ant_simu.cpp`) :

1. boucle sur $i \in [0, N)$: avance de la fourmi i en manipulant `x[i]`, `y[i]`, `state[i]`, `seed[i]` ;
2. évaporation globale des phéromones (`do_evaporation`) ;
3. validation/swap de la carte de phéromones (`update`).

La logique par fourmi reste volontairement identique à la version AoS (mêmes choix de voisin, mêmes branches, même condition de mouvement `consumed_time < 1`) afin de comparer les performances en isolant l’effet du layout mémoire.

3.1.6 Découplage de `SDL_Point` hors hot loop et adaptation du rendu

Le type `SDL_Point` est pratique pour le rendu mais il couple le noyau de calcul à SDL. Dans la stratégie SoA, le noyau manipule des scalaires `int32_t` ; la conversion vers un point SDL (si nécessaire) est repoussée au moment du rendu. Dans le même esprit, une surcharge `mark_pheronome(x, y)` côté `pheronome` permet de marquer les phéromones sans construire un `SDL_Point` à chaque pas.

Enfin, le `Renderer` est adapté pour accepter soit une population AoS (`std::vector<ant>`), soit une population SoA (`AntsSoA`). Le rendu itère alors :

- en AoS : sur les objets et `ant.get_position()` ;
- en SoA : sur l’indice i et les tableaux `x[i]`/`y[i]`.

3.1.7 Validation et protocole de comparaison

La validation repose sur la compilation en mode *release* puis l’exécution du mode `-benchmark` en forçant successivement `-layout aos` puis `-layout soa`. Le format des métriques `METRIC ...` est conservé afin de réutiliser le pipeline de consolidation existant (`scripts/measure_temps.sh`). La comparaison se focalise sur $K1$ (et donc $K0$), puisque la vectorisation vise prioritairement le noyau d’avance des fourmis.

3.2 Résultat du Profilage

Les résultats présentés ci-dessous proviennent de la campagne de mesure : `results/test_vectorisation_layout/soa/20260227_115411/summary.md`. La configuration utilisée est la suivante : layout `soa`, 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 6 – Temps one-shot d’initialisation (layout SoA)

Partie	ms moyen	écart-type
P0	44.024725	1.211201
P1	0.951707	0.044184
P2	0.072038	0.007817

TABLE 7 – Décomposition des coûts (profilage, layout SoA)

Partie	ms/iter moyen	écart-type	% kernel
K1	4.264492	0.022742	94.0300
K2	3.230802	0.017821	71.2376
K3	0.668624	0.005969	14.7428
K4	0.266004	0.004142	5.8653
K5	0.004469	0.000404	0.0985
R0	1.024001	0.013386	n/a
E0	0.002005	0.000105	n/a

TABLE 8 – Indicateurs globaux par répétition (layout SoA)

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	5.572491	4.550202	1.020098	1098852.270736
2	5.528406	4.507923	1.018436	1109158.323270
3	5.615466	4.574892	1.038652	1092921.937721
4	5.581273	4.540269	1.039038	1101256.219797
5	5.508628	4.502948	1.003779	1110383.627732
Moyenne	5.561253	4.535247	1.024001	1102514.475851
Écart-type	0.038248	0.026872	0.013386	6528.031897

Fraction parallélisable candidate (layout SoA) :

$$F = (K1 + K4)/K0 = 0.998952$$

TABLE 9 – Prédiction théorique selon Amdahl (layout SoA)

Threads	S(p)
2	1.997906
4	3.987463
8	7.941739
16	15.752373

3.3 Analyse des Résultats

Afin d’évaluer l’impact de la vectorisation SoA de manière contrôlée, nous comparons deux campagnes de mesure exécutées sur la même machine, avec les mêmes paramètres et le même script de profilage, à 43 s d’intervalle :

— AoS (baseline) : `results/test_vectorisation_layout/aos/20260227_115454/summary.md` ;

— SoA (vectorisé) : `results/test_vectorisation_layout/soa/20260227_115411/summary.md`.

La variation relative est définie par :

$$\Delta_{\%}(X) = 100 \times \frac{X_{\text{SoA}} - X_{\text{AoS}}}{X_{\text{AoS}}}.$$

TABLE 10 – Comparaison AoS vs SoA (moyennes sur 5 répétitions)

Métrique	AoS	SoA	Δ	$\Delta_{\%}$
T_{kernel} (ms/iter)	4.533697	4.535247	+0.001550	+0.034
$K1$ (ms/iter)	4.257748	4.264492	+0.006744	+0.158
$R0$ (ms/iter)	1.040353	1.024001	-0.016352	-1.572
$T_{\text{iter,total}}$ (ms/iter)	5.576019	5.561253	-0.014766	-0.265
fourmis/s	1102929.095441	1102514.475851	-414.619590	-0.038

1. **Pas de speedup mesurable sur le noyau (K0/K1).** Les temps noyau T_{kernel} ($\approx K0$) sont quasi identiques : AoS 4.533697 ± 0.037727 contre SoA 4.535247 ± 0.026872 ms/iter, soit $\Delta_{\%} = +0.034\%$ (légère dégradation SoA). De même, $K1$ passe de 4.257748 ± 0.030779 (AoS) à 4.264492 ± 0.022742 ms/iter (SoA), soit $\Delta_{\%} = +0.158\%$. Ces écarts sont très inférieurs aux dispersions inter-répétitions (écarts-types), ce qui ne permet pas de conclure à un gain significatif sur le noyau via le seul changement de layout des attributs des fourmis.
2. **La distribution des hotspots reste inchangée.** Dans les deux layouts, $K1$ représente environ 94% du temps noyau (AoS : 93.9134%, SoA : 94.0300%). Au sein de $K1$, $K2$ reste la composante dominante : il pèse environ 71% du noyau (AoS : 71.1255%, SoA : 71.2376%). Autrement dit, le coût principal demeure la logique interne d’avance des fourmis (branchements, choix aléatoires, condition `consumed_time<1`, et surtout accès/écritures indirectes sur `pheronome` et accès à `land`), plutôt que le stockage des attributs `x/y/state/seed`.
3. **Légère amélioration du rendu, sans impact sur le calcul.** Le coût de rendu $R0$ diminue de 1.040353 ± 0.015469 (AoS) à 1.024001 ± 0.013386 ms/iter (SoA), soit $\Delta_{\%} = -1.572\%$. Cette baisse explique la petite amélioration du temps total par itération ($\Delta_{\%} = -0.265\%$), alors même que le noyau de calcul reste inchangé. Le gain observé est donc principalement hors noyau.
4. **Débit quasi identique.** Le débit passe de $1102929.095441 \pm 9173.468667$ (AoS) à $1102514.475851 \pm 6528.031897$ fourmis/s (SoA), soit $\Delta_{\%} = -0.038\%$. Cette stabilité est cohérente avec l’absence de gain sur T_{kernel} , puisque le débit (tel que calculé par le script) dépend essentiellement du temps noyau.
5. **Interprétation et suite.** La vectorisation SoA constitue surtout une *restructuration* conforme au point 5.1 (tableaux par attribut, fourni repérée par indice) et un prérequis pour la suite (5.2 OpenMP, et potentielles transformations plus profondes). En revanche, ces mesures suggèrent qu’une amélioration de performance du noyau exigera d’attaquer la partie dominante $K2$ (accès mémoire sur la carte de phéromones, organisation des mises à jour, réduction de divergence du `while`, parallélisation), plutôt que le seul layout des attributs agents.

4 Parallélisation en mémoire partagée (OpenMP)

4.1 Stratégie de parallélisation

4.1.1 Objectif

Après la vectorisation (layout SoA), l'objectif du point OpenMP est de réduire le temps noyau $K0$ en parallélisant les boucles les plus coûteuses, tout en conservant une sortie benchmark reproductible (METRIC ...) pour l'analyse de speedup.

4.1.2 Choix d'architecture

L'implémentation OpenMP est portée par le backend `omp+soa` (`src/sim/omp_soa_backend.cpp`) et sélectionnée via :

- `-exec omp`
- `-layout soa`

4.1.3 Boucles parallélisées

1. **Évaporation des phéromones ($K4$).** `do_evaporation()` (`src/pheronome.hpp`) utilise `#pragma omp parallel for collapse(2) schedule(static)` sur le balayage de grille. Les cellules sont indépendantes pendant cette phase.
2. **Avance des fourmis ($K1$).** Dans `OmpSoaBackend::step`, la boucle sur les indices de fourmis est parallélisée avec `#pragma omp for schedule(static)`. Pendant la région parallèle, chaque thread :
 - met à jour ses fourmis locales (`x[i]`, `y[i]`, `state[i]`, `seed[i]`),
 - enregistre les cellules touchées (`touched_v1/touched_v2`),
 - accumule son `food_delta` local.

Aucune écriture globale directe dans la carte de phéromones n'est faite dans la boucle parallèle.

4.1.4 Correctness et absence de data races

Les points sensibles en mémoire partagée sont :

- la carte de phéromones (écritures concurrentes potentielles),
- le compteur global `food_quantity` (incréments concurrents).

Le schéma retenu évite ces conflits :

- structures locales par thread (`touched_v1/touched_v2`, `food_delta`),
- fusion après la région parallèle,
- replay séquentiel des marquages.

4.1.5 Réduction sparse et robustesse d'ordre

Après la région parallèle :

1. concaténation des vecteurs `touched` de tous les threads,
2. tri + `unique` pour ne garder qu'une fois chaque cellule,
3. application séquentielle des marques sur la carte globale.

Ce schéma reste cohérent avec un marquage idempotent (politique de fusion par max), robuste à l'ordre de traitement des fournis.

4.1.6 Profilage OpenMP-safe

Le profilage combine deux niveaux :

- **wall-time** aux frontières des kernels ($K1$, $K4$, $K5$),
- **work-time** pour $K2/K3$ (somme du travail interne des threads).

En OpenMP, $K2/K3$ sont accumulés localement par thread puis fusionnés après la région parallèle (pas d'atomiques dans la boucle chaude). Ainsi :

- $K1$ est un temps mur (comparaison directe de performance),
- $K2/K3$ décrivent la quantité de travail, mais ne doivent pas être lus comme temps mur.

4.2 Résultat du Profilage

4.2.1 Protocole expérimental

Campagnes utilisées :

- OpenMP SoA : `results/test_openmp_report_local/omp/soa/latest`
- Référence serial SoA (1 thread) : `results/test_openmp_report_local/serial/soa/latest`

Configuration commune :

- 1200 itérations, 200 warm-up, donc 1000 itérations mesurées,
- mode `-benchmark -no-render`,
- 7 répétitions par configuration,
- machine : AMD Ryzen 7 5800H (8 coeurs physiques / 16 threads logiques), WSL2,
- sweep $\{1, 2, 4, 8\}$ aligné sur les coeurs physiques.

Commande type :

```
./scripts/measure_openmp.sh --folder test_openmp_report_local --layout soa --exec omp \
  --threads 1,2,4,8 --iterations 1200 --warmup 200 --reps 7
```

La variation relative du temps noyau est définie par :

$$\Delta(K0_p) = K0_p - K0_1, \quad \Delta_{\%}(K0_p) = 100 \times \frac{K0_p - K0_1}{K0_1}$$

où $K0_1$ désigne la valeur mesurée avec 1 thread.

TABLE 11 – OpenMP SoA : temps noyau, variation relative et speedup selon le nombre de threads

Threads	K0 moyen	écart-type	Δ	$\Delta_{\%}$	Speedup $S(p)$	Efficacité $E(p) = S/p$
1	8.684056	0.264434	+0.000000	+0.000	1.000000	1.0000
2	5.539441	0.275826	-3.144615	-36.211	1.567677	0.7838
4	4.612187	0.373999	-4.071869	-46.889	1.882850	0.4707
8	3.564663	0.806521	-5.119393	-58.952	2.436151	0.3045

4.2.2 Calcul explicite du speedup mesuré

Le speedup expérimental est défini par :

$$S(p) = \frac{T(1)}{T(p)}$$

où $T(p)$ est le temps moyen de $K0$ avec p threads. Dans cette section, la référence $T(1)$ est **le backend omp+soa à 1 thread** (pas **serial+soa**), afin de mesurer strictement l'effet du threading à algorithme constant.

$$T(1) = 8.684056, \quad T(2) = 5.539441, \quad T(4) = 4.612187, \quad T(8) = 3.564663$$

$$S(1) = \frac{8.684056}{8.684056} = 1.000000$$

$$S(2) = \frac{8.684056}{5.539441} = 1.567677321 \approx 1.567677$$

$$S(4) = \frac{8.684056}{4.612187} = 1.882850308 \approx 1.882850$$

$$S(8) = \frac{8.684056}{3.564663} = 2.436151334 \approx 2.436151$$

4.2.3 Définition et lecture de l'efficacité

L'efficacité est définie par :

$$E(p) = \frac{S(p)}{p}$$

Une efficacité proche de 1 signifie que l'augmentation du nombre de threads est bien convertie en accélération utile. À l'inverse, lorsque $E(p)$ diminue, cela signifie qu'une partie du gain potentiel est perdue à cause des overheads, de la bande passante mémoire ou des parties encore partiellement séquentielles. En OpenMP, cette baisse d'efficacité vient surtout du caractère *memory-bound* de $K4$ et de la phase de fusion sparse qui limite la scalabilité de $K1$.

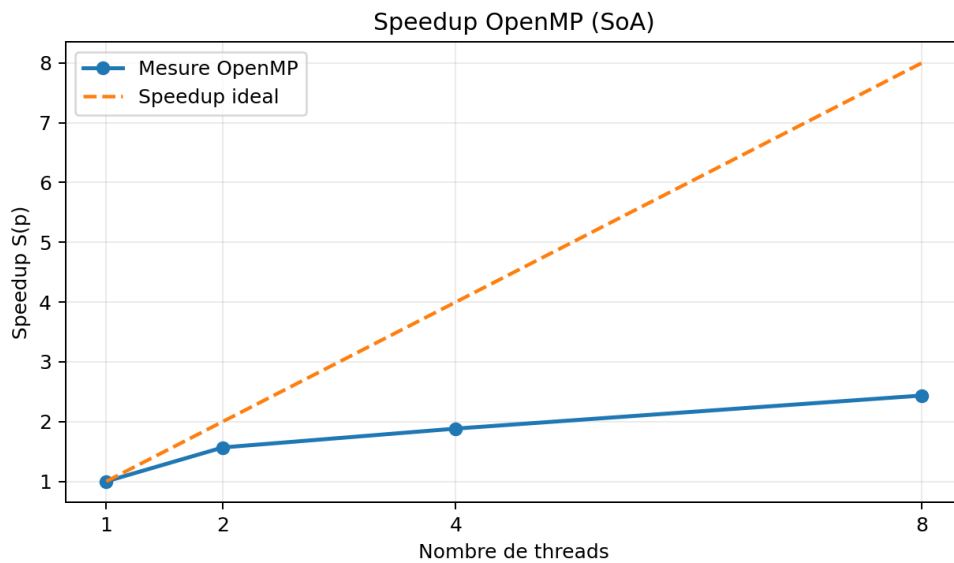


FIGURE 1 – Speedup mesuré OpenMP (SoA) vs speedup idéal linéaire

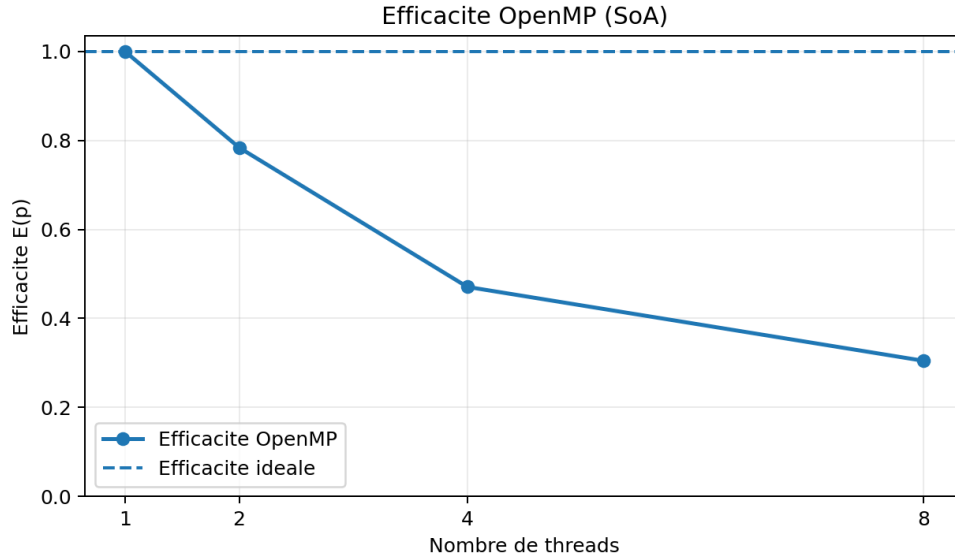


FIGURE 2 – Efficacité OpenMP (SoA) en fonction du nombre de threads

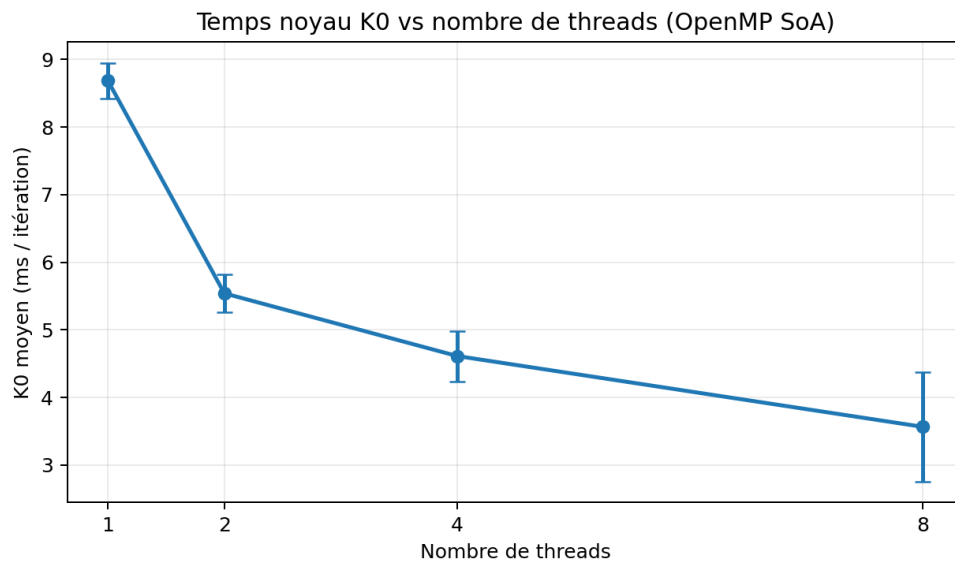
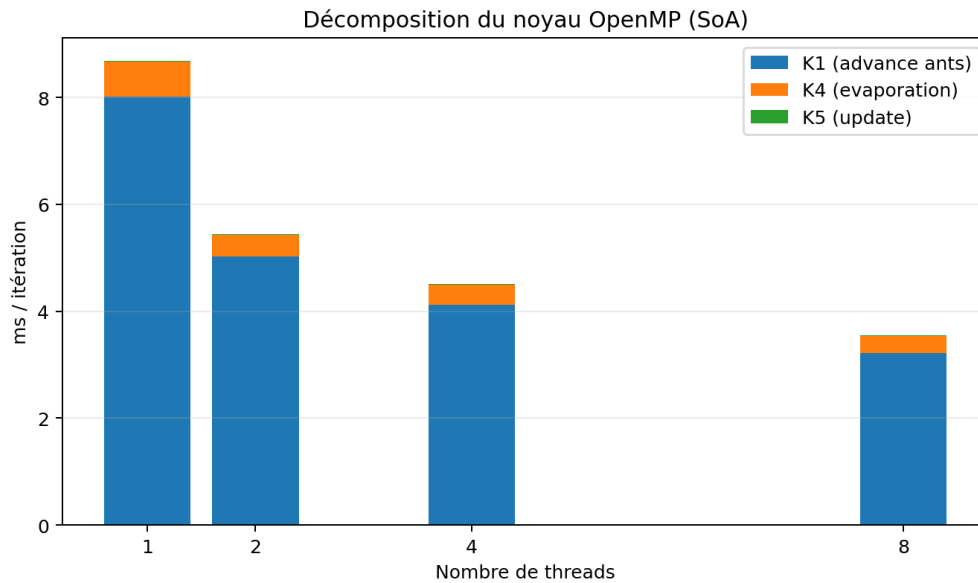
FIGURE 3 – Temps noyau $K0$ en fonction du nombre de threads (barres d'erreur : écart-type)

TABLE 12 – Décomposition du noyau OpenMP SoA (wall-time)

Threads	K1 (ms/iter)	K4 (ms/iter)	K5 (ms/iter)
1	8.014997	0.649239	0.015685
2	5.023505	0.400604	0.011876
4	4.126514	0.366018	0.014365
8	3.219344	0.317306	0.014992

FIGURE 4 – Décomposition $K1/K4/K5$ du noyau OpenMP SoATABLE 13 – Work-time interne OpenMP SoA ($K2$, $K3$)

Threads	K2 work-time (ms/iter)	K3 work-time (ms/iter)
1	3.640941	0.532093
2	3.275849	0.587566
4	3.503250	0.777031
8	3.399851	0.815067

4.2.4 Vérifications empiriques de correctitude

En complément de l’argument “absence de races”, on vérifie deux invariants simples :

- `food_quantity` monotone non décroissante (`food_monotonic_ok=1`),
- stabilité d’indicateurs globaux de phéromones entre 1 et 8 threads (même seed).

TABLE 14 – Checks de correctitude OpenMP (5000 itérations, warmup 500, seed 2026)

Métrique	OMP 1 thread	OMP 8 threads
<code>food_monotonic_ok</code>	1	1
<code>food_quantity_final</code>	1556	1556
<code>phen_v1_sum</code>	2989.08	2989.08
<code>phen_v2_sum</code>	16321.2	16321.2
<code>phen_v1_max</code>	1.0	1.0
<code>phen_v2_max</code>	1.0	1.0

4.3 Analyse des Résultats

Le speedup croît bien avec le nombre de threads, mais reste loin de la droite idéale :

$$S(8) = 2.4362 \quad (\text{vs idéal } 8.0)$$

- **Lecture Amdahl.** Avec $S(8) = 2.436151$, on obtient :

$$f \approx \frac{8/S(8) - 1}{7} \approx 0.33, \quad S_{\max} \approx \frac{1}{f} \approx 3.06.$$

Le plafonnement observé autour de 2.4 est cohérent avec cette borne.

- **K4 (évaporation), comportement memory-bound.**

$$K4 : 0.649239 \rightarrow 0.317306 \text{ ms/iter } (1 \rightarrow 8) \approx -51.1\%.$$

Le gain est net mais non linéaire, typique d'une limite de bande passante mémoire.

- **Lecture de l'efficacité.** La Figure 2 et la Table 11 montrent que l'efficacité décroît avec le nombre de threads ($E(2) = 0.7838$, $E(4) = 0.4707$, $E(8) = 0.3045$). Cette baisse est cohérente avec les deux limitations principales déjà observées : la partie K4, contrainte par la mémoire, et la fusion post-parallèle de K1, qui conserve une partie séquentielle non négligeable.

- **K1 (avance des fourmis), gain significatif mais borné.**

$$K1 : 8.014997 \rightarrow 3.219344 \text{ ms/iter } (1 \rightarrow 8) \approx -59.8\%.$$

La phase de fusion sparse (concat, sort+unique, replay) reste partiellement séquentielle et freine la scalabilité.

- **Interprétation de K2/K3.** Les valeurs K2/K3 sont des work-times : elles décrivent le volume de travail interne et l'évolution de la granularité, mais ne s'additionnent pas directement en temps mur.
- **Comparaison serial+soa vs omp+soa à 1 thread.** La référence serial locale vaut $K0 = 7.924494$ ms/iter, contre 8.684056 ms/iter pour omp+soa à 1 thread (+9.6%). Cet écart vient du coût structurel propre au backend OMP (buffers touched, merge, dedup, replay), indépendamment du speedup de threading.

4.4 Interprétation

Les Tables 11, 12 et 13, ainsi que les Figures 1, 2, 3 et 4, montrent une image globalement cohérente.

La parallélisation OpenMP apporte un gain réel sur cette machine : la variation relative de K0 devient de plus en plus négative lorsque le nombre de threads augmente, jusqu'à -58.95% à 8 threads, ce qui correspond à un speedup de 2.44. Autrement dit, le backend omp+soa accélère bien, même si l'on reste loin du speedup linéaire idéal.

La table de décomposition confirme ensuite que le bénéfice vient principalement de la baisse de K1, tandis que K4 reste limité par la bande passante mémoire et que la phase de fusion sparse empêche une scalabilité parfaite. Enfin, les valeurs K2/K3 rappellent que la quantité de travail interne ne disparaît pas : OpenMP réduit surtout le temps mur, pas le volume total de travail réalisé par l'ensemble des threads.

5 Parallélisation du code (MPI) — Première façon

5.1 Stratégie de parallélisation

5.1.1 Objectif

L'objectif de cette étape est de réduire le temps noyau $K0$ via une exécution distribuée (MPI), puis de mesurer l'accélération en fonction du nombre de processus. On conserve la même logique expérimentale que pour OpenMP : mode benchmark, no-render, plusieurs répétitions, et calcul du speedup.

5.1.2 Choix d'architecture

Dans notre implémentation `mpi1`, on ajoute un backend dédié `mpi1+soa` (sélectionné via `-exec mpi1 -layout soa`). Les modes existants `serial` et `omp` sont conservés tels quels (même comportement, même logique de benchmark). Le code MPI est ainsi isolé dans un chemin d'exécution spécifique, afin d'éviter toute régression sur les autres modes.

5.1.3 Répartition du travail

Le principe de la “première façon” est :

- chaque processus MPI possède une copie complète de l'environnement (terrain, obstacles, phéromones $V1/V2$, nid, nourriture) ;
- les fourmis sont réparties par indices (bloc par rang `rank`) ;
- chaque rang simule uniquement sa plage locale de fourmis.

Ce choix est simple à implémenter et ne requiert pas de migration de fourmis entre rangs.

5.1.4 Communications et synchronisation

Dans `mpi1`, on conserve :

- `MPI_Allreduce(..., MPI_MAX)` pour fusionner les cartes $V1/V2$;
- `MPI_Allreduce(..., MPI_SUM)` pour agréger le `food_delta` (quantité de nourriture collectée) et maintenir une métrique globale cohérente.

Contrôle de granularité. La version actuelle ajoute un paramètre de granularité :

- `-mpi-sync-every K` (par défaut $K = 1$) ;
- la fusion $V1/V2$ est effectuée une fois toutes les K itérations.

Le compteur de nourriture reste synchronisé à *chaque itération* (payload faible), et la sortie benchmark est imprimée uniquement par le rang 0 (root-only), afin de préserver un format `METRIC ...` propre et facilement parsable.

5.1.5 Correctness et cohérence

MPI travaille en mémoire distribuée : chaque rang a son propre espace mémoire. On évite donc les data races de type mémoire partagée entre processus. La cohérence globale est assurée par les collectives :

- **MAX** pour la fusion des phéromones, cohérent avec un ordre de traitement potentiellement arbitraire des fourmis entre rangs ;
- **SUM** pour maintenir un compteur global de nourriture cohérent.

5.1.6 Profilage et métriques MPI

En MPI, le temps global d’une itération correspond au temps du rang le plus lent. Pour chaque métrique temporelle ($K0$, $K1$, $K4$, $K5$, et $K_{\text{MPI_sync}}$), on retient donc :

$$T_p = \max_{r \in \{0, \dots, p-1\}} T_p^{(r)}$$

via `MPI_Allreduce(..., MPI_MAX)` sur les temps locaux. Ce choix correspond aussi à l’idée de *load balancing* : le temps parallèle est pénalisé par le processus le plus lent, et toute variabilité (bruit système, déséquilibre) se répercute sur le temps global.

On introduit $K_{\text{MPI_sync}}$ pour isoler le coût des synchronisations collectives (`Allreduce`) et interpréter la scalabilité.

5.2 Résultat du Profilage

5.2.1 Protocole expérimental

Campagne utilisée dans cette section :

— `results/test_mpi1_sync_every_rerun/20260307_155513`

Commande type :

```
OMP_NUM_THREADS=1 OMP_DYNAMIC=FALSE \
mpirun --bind-to core --map-by core -np P ./src/ant_simu.exe \
  --exec mpi1 --layout soa --benchmark --no-render \
  --iterations 400 --warmup 100 --mpi-sync-every K
```

Balayages réalisés :

$$P \in \{1, 2, 4, 8\}, \quad K \in \{1, 5, 10\}$$

avec **1 coeur par processus**.

Paramètres de campagne :

- répétitions : `reps = 3`;
- warmup : 100 itérations ;
- itérations mesurées : 300 (sur 400 totales) ;
- machine : AMD Ryzen 7 5800H, WSL2, Open MPI 4.1.6.

Le speedup est calculé à $P = 1$ **pour chaque valeur de K** :

$$S_K(p) = \frac{T_K(1)}{T_K(p)}, \quad E_K(p) = \frac{S_K(p)}{p}$$

Remarque sur la mesure du speedup. Le cours rappelle que l’algorithme séquentiel peut différer de l’algorithme parallèle (et que la mesure du speedup n’est alors pas “évidente”). Ici, nous prenons comme référence **MPI1** à $P = 1$ (pour une valeur donnée de K), afin de comparer un *même algorithme* et d’isoler l’effet du parallélisme MPI. La comparaison avec `serial+soa` reste utile comme contrôle, mais n’est pas utilisée comme référence principale du speedup MPI1.

5.2.2 Scénario de scalabilité (cours)

Nous évaluons ici le *worst speedup* : la quantité de données *globale* est fixée (nombre total de fourmis, taille de la carte) et on augmente uniquement le nombre de processus P . Dans un scénario *best speedup*, on conserverait une quantité de données fixe *par processus* (en augmentant par exemple le nombre total de fourmis avec P), ce qui tend à améliorer le speedup.

5.2.3 Baseline MPI1 ($K = 1$, sync à chaque itération)

La variation relative du temps noyau est définie par :

$$\Delta(K0_p) = K0_p - K0_1, \quad \Delta_{\%}(K0_p) = 100 \times \frac{K0_p - K0_1}{K0_1}$$

où $K0_1$ désigne la valeur mesurée avec 1 processus.

TABLE 15 – MPI1 SoA ($K = 1$) : temps noyau, variation relative et speedup selon le nombre de processus

np	K0 moyen	écart-type	Δ	$\Delta_{\%}$	Speedup $S(p)$	Efficacité $E(p)$
1	6.632929	0.069502	+0.000000	+0.000	1.000000	1.000000
2	5.875666	0.309316	-0.757263	-11.417	1.128881	0.564441
4	9.602010	0.587313	+2.969081	+44.763	0.690786	0.172696
8	19.178292	0.892592	+12.545363	+189.138	0.345856	0.043232

TABLE 16 – Décomposition du noyau MPI1 SoA ($K = 1$)

np	K1 (ms/iter)	K4 (ms/iter)	K5 (ms/iter)	$K_{\text{MPI_sync}}$ (ms/iter)
1	6.304193	0.312222	0.007008	0.006936
2	3.709763	0.694195	0.013545	1.571793
4	2.523657	1.906690	0.040042	5.304413
8	1.647622	4.160358	0.101536	13.420741

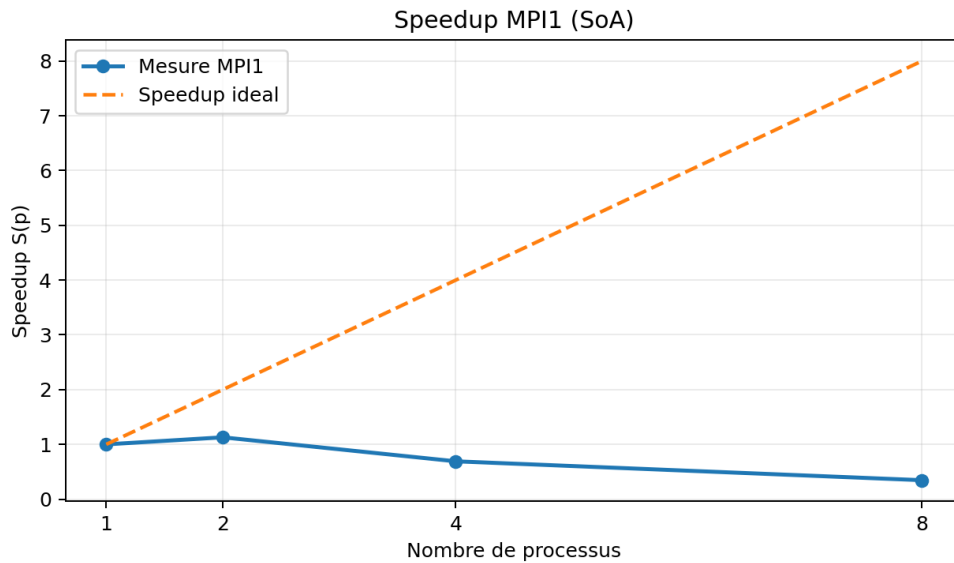


FIGURE 5 – Speedup mesuré MPI1 (SoA) pour $K = 1$ vs speedup idéal linéaire

5.2.4 Définition et lecture de l'efficacité

L'efficacité est définie par :

$$E(p) = \frac{S(p)}{p}$$

Une efficacité proche de 1 signifie que les processus supplémentaires sont utilisés de manière efficace. Lorsqu'elle diminue, cela indique qu'une part croissante du temps est perdue dans les overheads de parallélisation. En MPI1, cette baisse d'efficacité vient surtout du coût des synchronisations globales et des communications collectives sur les cartes complètes V1/V2.

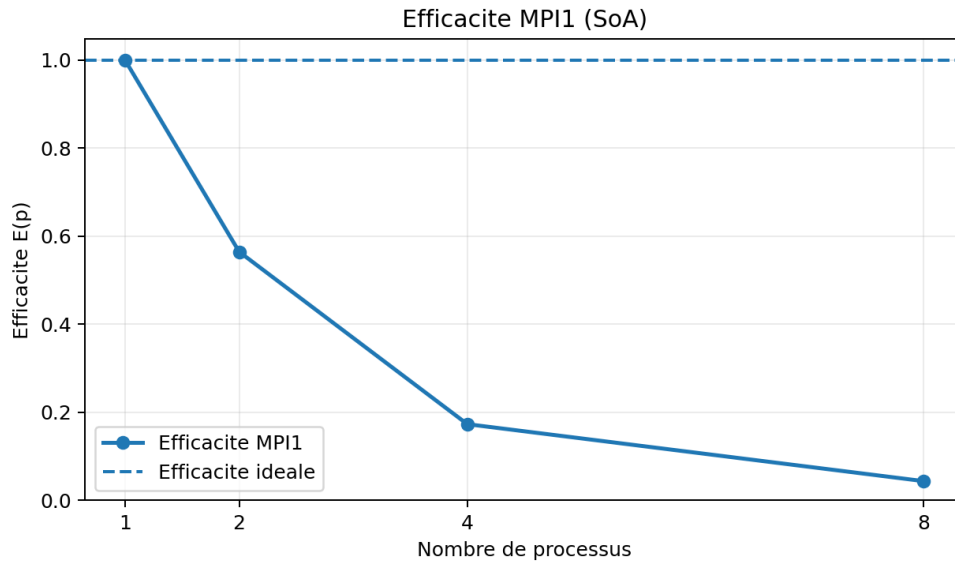


FIGURE 6 – Efficacité MPI1 (SoA) en fonction du nombre de processus

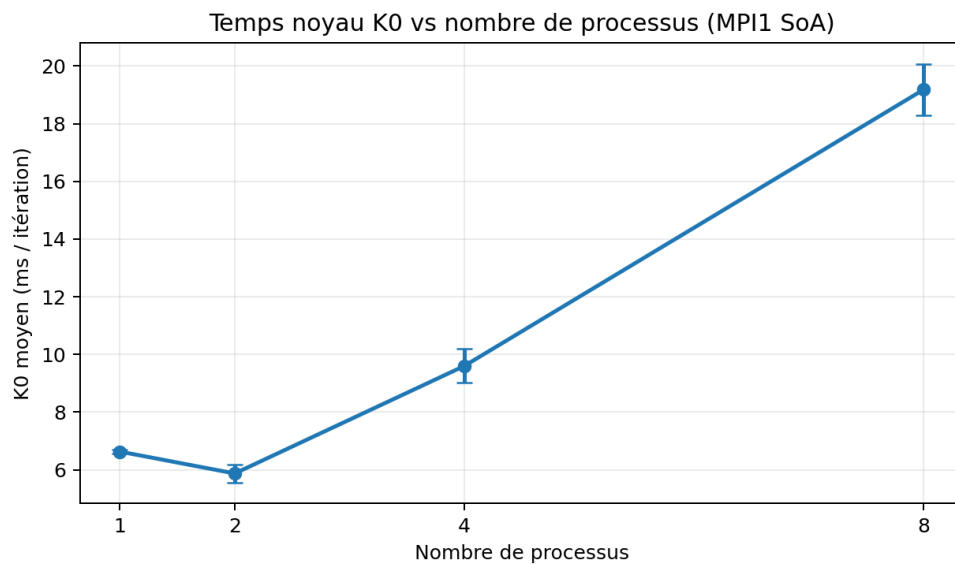
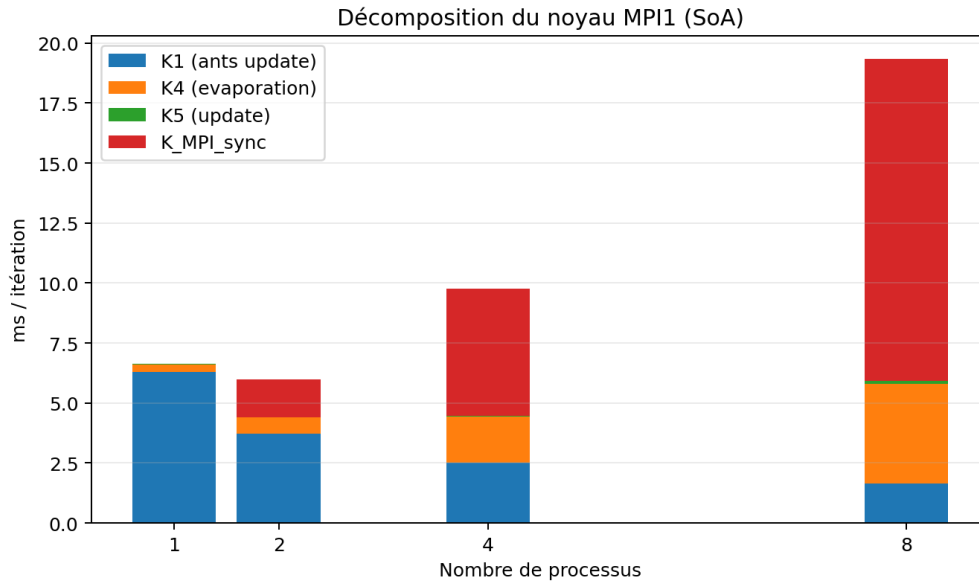


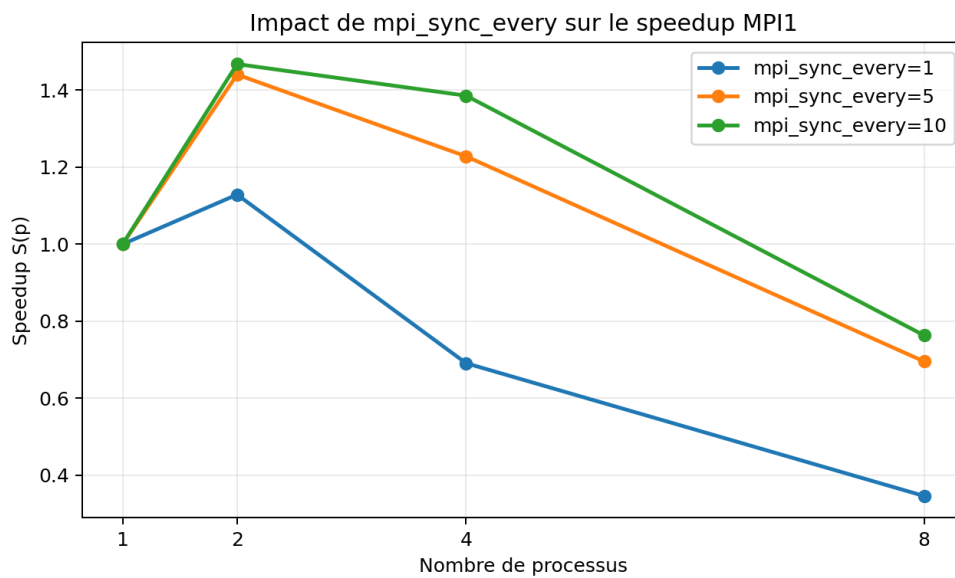
FIGURE 7 – Temps noyau $K0$ en fonction du nombre de processus ($K = 1$)

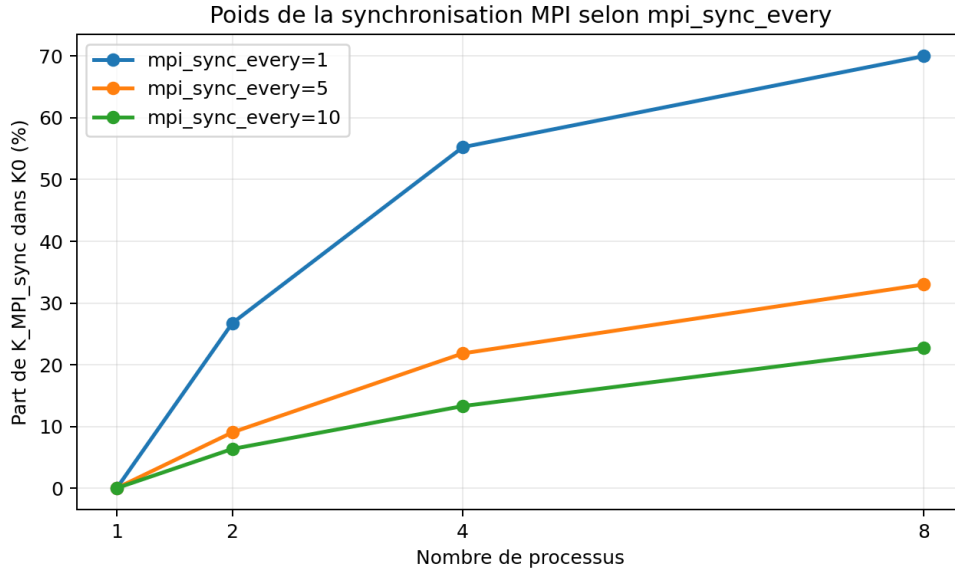
FIGURE 8 – Décomposition $K1/K4/K5/K_{MPI_sync}$ du noyau MPI1 SoA ($K = 1$)

5.2.5 Impact de mpi_sync_every

TABLE 17 – Impact de la fréquence de synchronisation sur la scalabilité MPI1

K	$S_K(2)$	$S_K(4)$	$S_K(8)$	Part K_{MPI_sync} à $np = 8$
1	1.128881	0.690786	0.345856	69.98%
5	1.440629	1.228152	0.695220	33.02%
10	1.467855	1.385889	0.763005	22.74%

FIGURE 9 – Effet de `mpi_sync_every` sur le speedup MPI1

FIGURE 10 – Part de $K_{\text{MPI_sync}}$ dans K_0 selon mpi_sync_every

5.3 Analyse des Résultats

5.3.1 Comportement global et scalabilité

Avec la baseline $K = 1$, on observe :

- un petit gain à $np = 2$ ($S(2) = 1.13$) ;
- une dégradation nette ensuite ($S(4) = 0.69$, $S(8) = 0.35$).

5.3.2 Lecture “granularité” et coût de communication

Le cours rappelle qu’un échange de messages comporte (i) un coût initial (constant) et (ii) un coût de transfert proportionnel à la quantité de données. Dans MPI1, la fusion des phéromones utilise des collectives **Allreduce(MAX)** sur les **cartes complètes** $V1/V2$, ce qui implique un volume de données élevé à chaque synchronisation. Ainsi, lorsque np augmente, le coût de communication et de synchronisation peut dominer le temps total, ce qui limite (voire inverse) le speedup.

La cause principale est bien $K_{\text{MPI_sync}}$, qui devient dominant quand np augmente : sa part passe d’environ 0.10% à $np = 1$ à 69.98% à $np = 8$. Dans cette “première façon”, on échange les cartes complètes $V1/V2$; le coût de synchronisation croît donc rapidement avec le nombre de processus.

5.3.3 Lecture de l’efficacité

La Figure 6 et la Table 15 montrent que l’efficacité chute très vite avec np dans la baseline $K = 1$ ($E(2) = 0.5644$, $E(4) = 0.1727$, $E(8) = 0.0432$). Cette baisse est cohérente avec la domination progressive de $K_{\text{MPI_sync}}$. Autrement dit, même si le calcul local sur les fourmis est mieux réparti, le coût des synchronisations globales absorbe une part de plus en plus grande du gain potentiel. Les essais avec $K > 1$ améliorent partiellement cette efficacité, ce qui confirme que le principal verrou est bien la communication.

5.3.4 Pourquoi $K4$ augmente avec np (régime *memory-bound*)

Même si MPI est un modèle “mémoire distribuée”, nos expériences sont réalisées sur une seule machine. Quand np augmente, plusieurs processus balaient la carte de phéromones en même temps, ce qui peut saturer la bande passante mémoire et dégrader $K4$ (régime *memory-bound*). Ce comportement est cohérent avec les limites liées aux accès mémoire : un balayage de grille parallélise le calcul, mais peut rester limité par la mémoire.

5.3.5 Effet de `mpi_sync_every` (trade-off cohérence / performance)

L’ajout de `-mpi-sync-every K` confirme l’analyse “granularité vs communication” :

- en réduisant la fréquence des `Allreduce(MAX)`, la part de $K_{\text{MPI_sync}}$ diminue fortement (à $np = 8$: 69.98% $\rightarrow$ 33.02% $\rightarrow$ 22.74% pour $K = 1, 5, 10$) ;
- la scalabilité s’améliore sensiblement ($S_5(4) = 1.23$, $S_{10}(4) = 1.39$) ;
- à $np = 8$, le speedup reste inférieur à 1 sur cette machine, mais la régression est nettement réduite (0.35 $\rightarrow$ 0.70 $\rightarrow$ 0.76).

En termes simples, `mpi_sync_every` augmente la granularité en réduisant le nombre d’échanges globaux : moins de synchronisations $\Rightarrow$ moins de coût de communication par itération. En contrepartie, la cohérence globale des cartes $V1/V2$ est moins fréquente (une fois toutes les K itérations), ce qui correspond à un compromis performance / cohérence.

5.3.6 Remarque sur le temps MPI (wall-time)

Le temps rapporté est un *wall-time* MPI (maximum entre rangs). Toute variabilité (déséquilibre léger, bruit système WSL2) est donc payée globalement, ce qui peut amplifier $K0$ lorsque np augmente.

5.4 Interprétation

Les résultats des Tables 15, 16 et 17, ainsi que des Figures 5, 6, 8, 9 et 10, conduisent à une conclusion assez nette.

La “première façon” MPI fonctionne correctement du point de vue algorithmique : la répartition des fourmis par rang est bien opérationnelle, la cohérence globale des cartes de phéromones est maintenue par `Allreduce(MAX)`, et les temps sont interprétés correctement comme des *wall-times* MPI via un maximum entre rangs. Autrement dit, l’implémentation est valide comme baseline distribuée.

En revanche, les résultats montrent aussi que cette stratégie scale mal lorsque le nombre de processus augmente. La Table 15 et la Figure 5 montrent qu’avec une synchronisation à chaque itération ($K = 1$), le gain est très limité à $np = 2$, puis devient une régression à $np = 4$ et $np = 8$. La Table 16 et la Figure 8 expliquent pourquoi : $K_{\text{MPI_sync}}$ devient progressivement la composante dominante du coût total.

Le paramètre `mpi_sync_every` confirme directement l’interprétation issue du cours sur la granularité. Lorsque l’on réduit la fréquence des synchronisations globales, le poids de $K_{\text{MPI_sync}}$ diminue fortement et le speedup s’améliore nettement, comme on le voit dans la Table 17 et les Figures 9 et 10. Cela montre que, dans notre cas, la limite principale n’est pas le calcul local sur les fourmis, mais bien le coût des communications collectives sur les cartes globales $V1/V2$.

La lecture finale est donc la suivante : MPI1 constitue une première version cohérente et simple à analyser, mais son coût de synchronisation globale devient trop élevé dès que np augmente.

Cette campagne justifie donc naturellement la transition vers la “deuxième façon” du projet, c’est-à-dire une décomposition de domaine avec échanges plus locaux, afin de réduire le volume de communication globale par itération.

6 Parallélisation du code (MPI) — Deuxième façon

6.1 Stratégie de parallélisation

6.1.1 Objectif

L'objectif de cette étape est d'implémenter la “deuxième façon” du sujet : au lieu de répliquer toute la carte de phéromones sur chaque rang (MPI1), on répartit la grille par sous-domaines, avec échanges locaux entre voisins. Le but est de réduire le coût de communication globale et d'améliorer la scalabilité en fonction du nombre de processus.

6.1.2 Choix d'architecture

Dans notre implémentation, le mode `mpi2` est porté par un backend séparé `mpi2+soa`, sélectionné via `-exec mpi2 -layout soa`. Les modes `serial`, `omp` et `mpi1` restent inchangés. Cette séparation permet d'analyser MPI2 sans introduire de régression sur les modes déjà validés.

6.1.3 Répartition du travail et données locales

Nous utilisons une décomposition 1D par lignes :

- chaque rang possède un bloc contigu de lignes $y_0 \dots y_1$,
- les voisins MPI sont **up** et **down**,
- chaque rang stocke une carte locale $V1/V2$ avec halo de 1 cellule.

Les fourmis gardent leurs coordonnées globales (x, y) , mais ne sont traitées localement que si leur position appartient au sous-domaine du rang.

6.1.4 Communications

MPI2 remplace les réductions globales de carte par des échanges locaux :

- **halo exchange** (`MPI_Sendrecv`) pour les lignes frontière de $V1/V2$,
- **migration de fourmis** via buffers agrégés (count + tableau POD) entre voisins.

Le compteur de nourriture est agrégé avec `MPI_Allreduce(SUM)`. Les temps MPI sont reportés en wall-time via `MPI_Allreduce(MAX)`. La sortie benchmark est root-only (rang 0), comme pour MPI1.

6.1.5 Politique de migration (baseline)

Le noyau de déplacement peut faire plusieurs sous-pas par itération (`consumed_time < 1.0`). Dans la baseline MPI2, nous utilisons une migration différée : si une fourmi sort du sous-domaine, elle est envoyée au voisin mais ne continue pas ses sous-pas restants pendant la même itération. Cette approximation est explicitement documentée pour rester lisible en cadre projet.

6.2 Résultat du Profilage

6.2.1 Protocole expérimental

Campagne utilisée :

— results/test_mpi2_report_v1/mpi2/soa/latest

Commande type :

```
./scripts/measure_mpi2.sh --folder test_mpi2_report_v1 \
  --np 1,2,4,8 --iterations 400 --warmup 100 --reps 3
```

Configuration :

- 400 itérations, 100 warm-up, donc 300 itérations mesurées,
- 3 répétitions par configuration,
- OMP_NUM_THREADS=1, OMP_DYNAMIC=FALSE,
- binding MPI : -bind-to core -map-by core,
- machine : AMD Ryzen 7 5800H (WSL2).

La variation relative du temps noyau est définie par :

$$\Delta(K0_p) = K0_p - K0_1, \quad \Delta_{\%}(K0_p) = 100 \times \frac{K0_p - K0_1}{K0_1}$$

où $K0_1$ est la valeur à $np = 1$.

TABLE 18 – MPI2 SoA : temps noyau, variation relative et speedup selon le nombre de processus

np	K0 moyen	écart-type	Δ	$\Delta_{\%}$	Speedup $S(p)$	Efficacité $E(p)$
1	16.328744	0.287091	+0.000000	+0.000	1.000000	1.000000
2	9.678549	0.032995	-6.650195	-40.727	1.687107	0.843553
4	5.755498	0.044726	-10.573246	-64.752	2.837069	0.709267
8	3.009207	0.042521	-13.319537	-81.571	5.426261	0.678283

6.2.2 Définition et lecture de l'efficacité

Le speedup est défini par :

$$S(p) = \frac{T(1)}{T(p)}$$

et l'efficacité par :

$$E(p) = \frac{S(p)}{p}$$

Une efficacité proche de 1 signifie que les processus supplémentaires sont bien utilisés. Quand $E(p)$ baisse, cela indique qu'une partie du gain est absorbée par les overheads (communication, synchronisation, déséquilibre éventuel). Ici, la baisse reste modérée entre 2, 4 et 8 processus.

TABLE 19 – Décomposition du noyau MPI2 SoA

np	K1	K4	K5	$K_{\text{MPI_sync}}$	$K_{\text{MPI_halo}} + K_{\text{MPI_migrate}}$
1	4.015616	2.006964	10.290746	0.007818	0.002027
2	2.063510	1.093025	5.741279	1.100832	0.089955
4	1.261674	0.593487	3.241676	0.959701	0.138488
8	0.679745	0.322101	1.726835	0.649644	0.094833

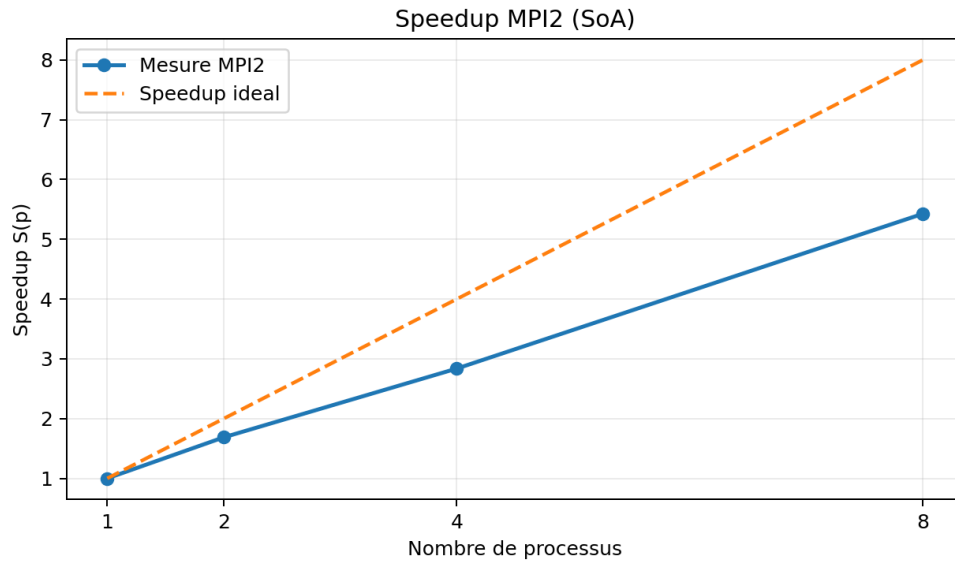


FIGURE 11 – Speedup mesuré MPI2 (SoA) vs speedup idéal linéaire

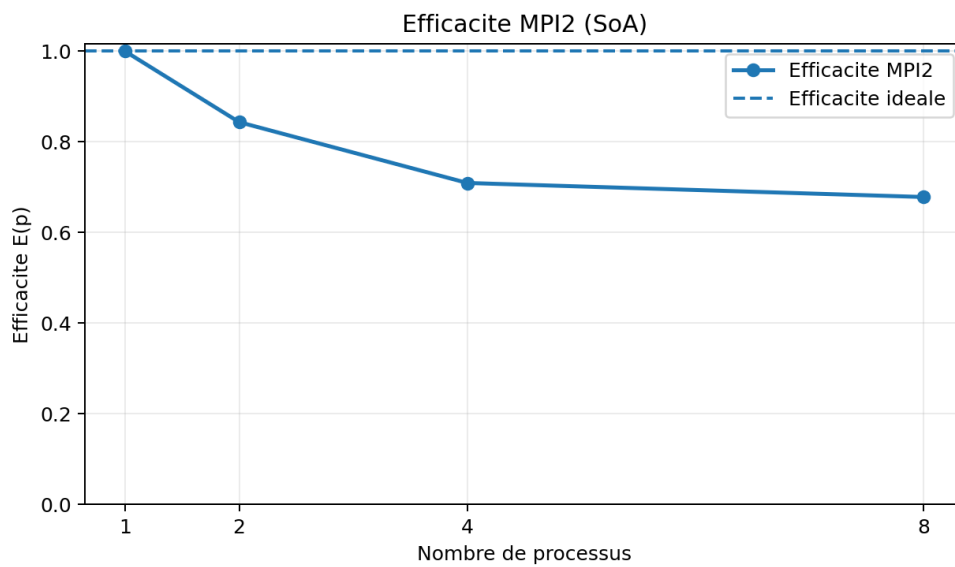
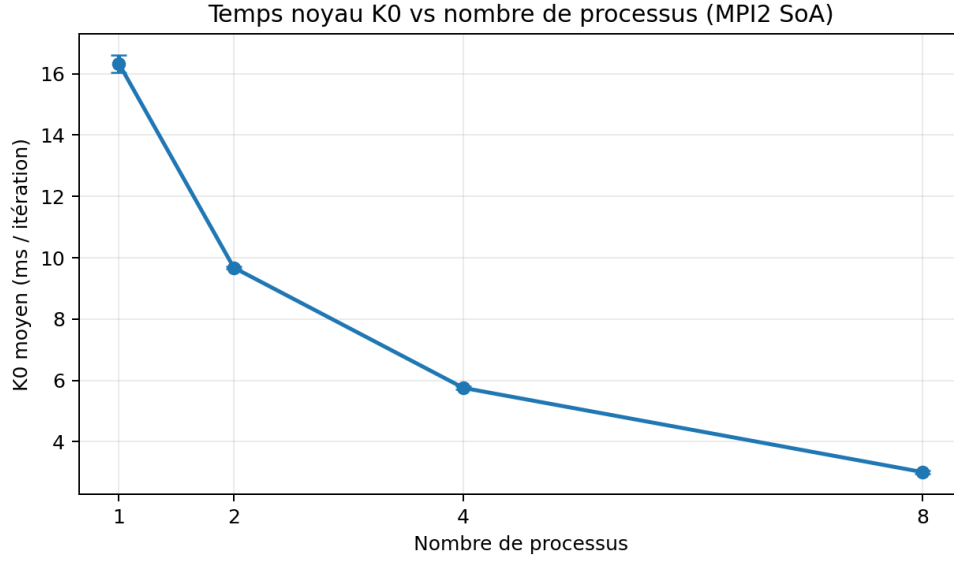
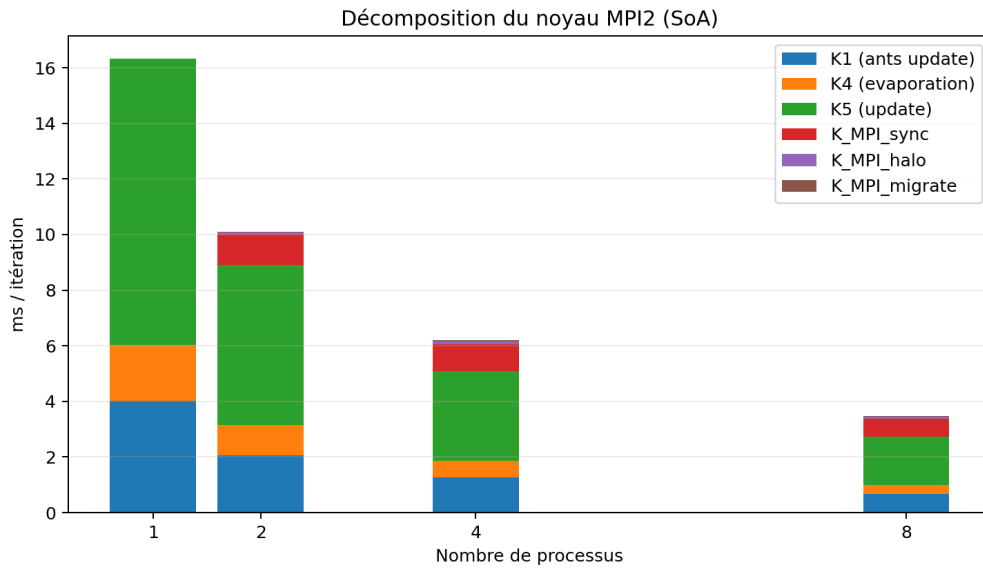


FIGURE 12 – Efficacité MPI2 (SoA) en fonction du nombre de processus

FIGURE 13 – Temps noyau K_0 en fonction du nombre de processus (MPI2 SoA)FIGURE 14 – Décomposition $K_1/K_4/K_5/K_{\text{MPI_sync}}/K_{\text{MPI_halo}}/K_{\text{MPI_migrate}}$

6.3 Analyse des Résultats

6.3.1 Scalabilité globale

Les résultats montrent une amélioration nette de K_0 quand np augmente (Table 18) :

$$S(2) = 1.687, \quad S(4) = 2.837, \quad S(8) = 5.426$$

Le speedup reste sous la droite idéale, mais la tendance est clairement scalable pour cette campagne.

6.3.2 Lecture Amdahl

Avec $S(8) = 5.426$, une estimation simple donne :

$$f \approx \frac{8/S(8) - 1}{7} \approx 0.068, \quad S_{\max} \approx \frac{1}{f} \approx 14.76$$

Ce calcul indique une fraction effectivement séquentielle faible dans cette version MPI2.

6.3.3 Bottlenecks observés

La Table 19 montre que :

- $K5$ reste la composante principale du noyau, même si son coût baisse avec np ,
- $K_{\text{MPI_sync}}$ augmente en valeur absolue entre $np = 1$ et $np = 2$, mais reste contenu ensuite,
- les coûts halo+migration sont présents mais restent modestes face à $K5$.

En pratique, MPI2 évite le coût des réductions globales de cartes complètes (cas MPI1) et remplace ce coût par des communications locales de voisinage, plus favorables à la scalabilité.

6.3.4 Lecture de l'efficacité

La Figure 12 confirme une baisse classique de l'efficacité avec np , mais la baisse reste progressive :

$$E(2) = 0.844, \quad E(4) = 0.709, \quad E(8) = 0.678$$

Cette évolution est cohérente avec un compromis attendu en MPI : le calcul local se répartit bien, tandis que le poids relatif des communications augmente quand on ajoute des processus.

6.4 Interprétation

Les Tables 18 et 19, ainsi que les Figures 11, 12, 13 et 14, montrent que la “deuxième façon” atteint l'objectif principal du point MPI : améliorer la scalabilité en réduisant la communication globale.

Le point important est que cette amélioration ne vient pas d'une optimisation locale isolée, mais d'un changement de stratégie de communication : au lieu de synchroniser des cartes complètes par `Allreduce(MAX)` à chaque itération, on échange principalement des informations de bord (halo) et des fourmis migrantes entre voisins.

Dans ce cadre, les résultats obtenus sont cohérents avec la théorie du cours sur la granularité : en diminuant le volume de communication globale par itération, on obtient un meilleur ratio calcul/communication et donc un speedup plus élevé.